

AI 能力五級進化論

# 從筆記到第二大腦

用一個案例，走完 L1 → L5

---

個人知識庫的 AI 演進之路

## 你的知識，散在哪裡？



### LINE 收藏

朋友傳的好文章，存了就忘



### 瀏覽器書籤

收藏了 200 個，打開過 3 個



### Notion / 筆記

寫了開頭，沒有後續



### Google Drive

資料夾叫「新資料夾(3)」



### 手機截圖

截了螢幕，從來沒翻過



### AI 對話記錄

問了很多好問題，對話關掉就沒了

你一年讀過

300+

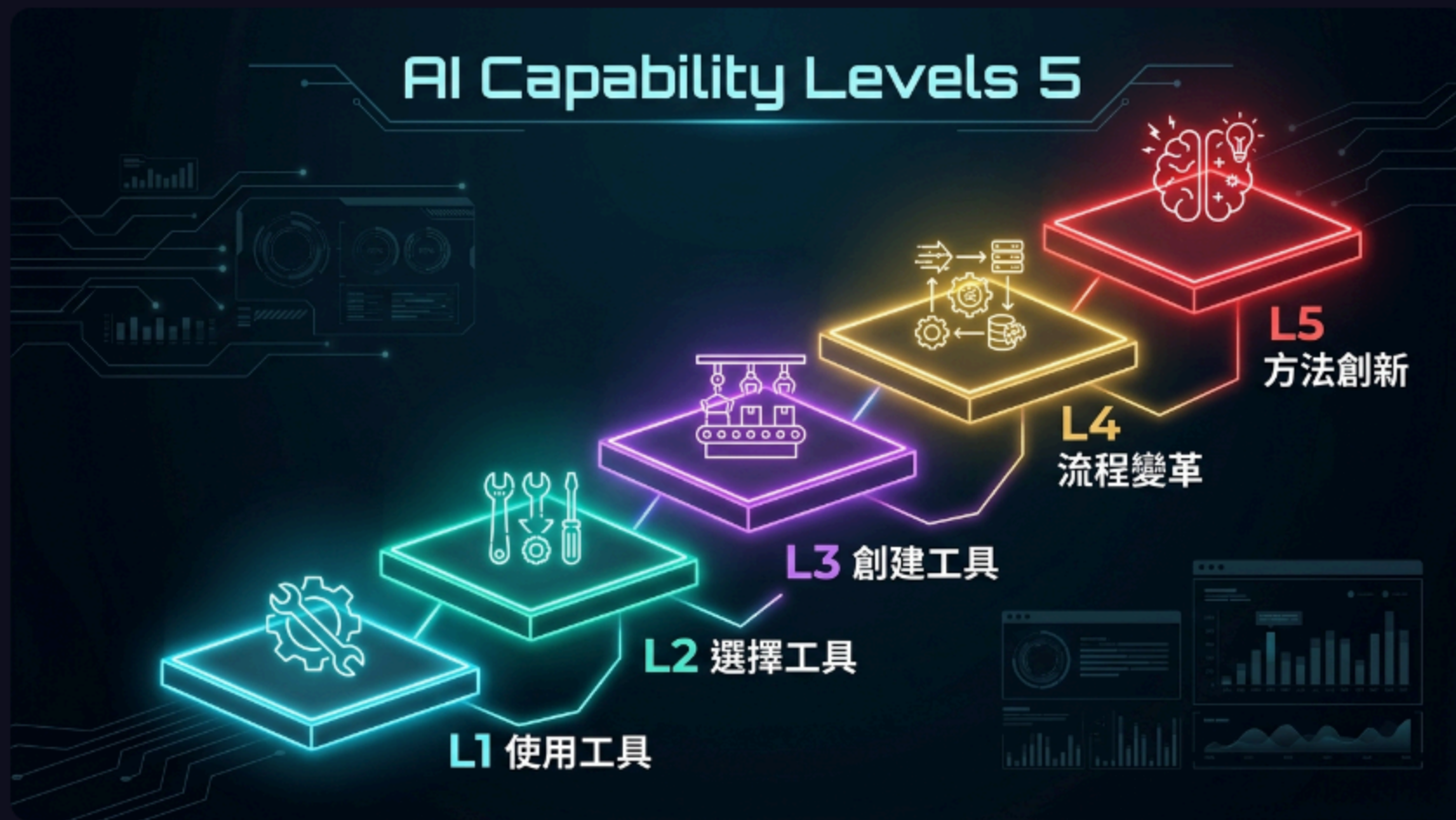
篇文章、報告、對話

—  
?

會拿來利用的，有多少？

知識管理的核心問題不是「存了多少」  
而是「用了多少」

# AI 能力五級 — 從點到立體再到創新



同一個邏輯，不同場景：ai-path 課程用「用→選→創→改造→陪跑」，大學演講用「點→線→面→立體→創新」

# OpenAI 五級智能 & Physical AI 核心突破



OpenAI 的五級是「AI 自身的能力演進」，我們的五級是「人駕馭 AI 的能力演進」— 兩條路線，交織前進。

接下來的旅程

## 同一個場景 — 個人知識庫

我們用「管理自己的知識」這一件事  
走一遍五級演進，看看每一級的差異、流程和突破點

L1 用工具



L2 選工具



L3 創工具



L4 流程改造



L5 方法創新

Level 1

# 用工具

點 — 一個工具做一件事

「AI 幫我整理了！」

## L1 場景 — AI 幫我整理一篇文章



每次都是一個獨立動作 — 一個工具、一個任務、一次完成。這就是「點」。



## L1 工具 — 一個人 AI 助手



### Custom GPT 「筆記整理助手」

預設好 Prompt 模板：

「摘要 300 字 + 3 個標籤 + 行動建議」

每次貼入文章就能用



### Gemini Gems 專屬問答

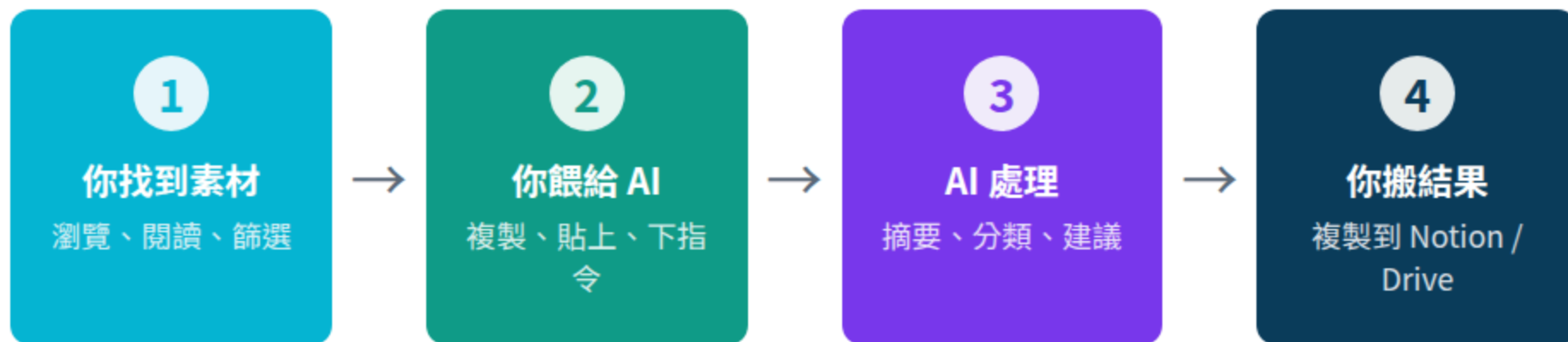
上傳你的筆記 PDF

變成可對話的知識庫

「這份報告的結論是什麼？」

L1 的核心：一個工具做一件事。你會用 AI 了，但每次都要手動啟動、手動搬結果。

## L1 你的角色 — 搬運工



你是每一步的「中繼站」— AI 很厲害，但要你一步步餵、一步步搬。

## L1 能力邊界

### ✓ L1 能做

- AI 摘要、翻譯、改寫
- 文章分類建議
- 快速問答
- 個人助手 (Custom GPT / Gems)

### ✗ L1 做不到

- 自動存檔到指定位置
- 工具之間自動串接
- 記住上次處理的內容
- 批量處理多篇文章

瓶頸：每次都要手動貼、手動搬。AI 很聰明，但你是唯一的搬運工。

# L1 突破點

突破前

我自己讀 + 自己整理  
速度慢、容易遺漏  
純靠意志力維持



突破後

AI 讀 + 我確認  
速度快、品質穩定  
但仍需手動啟動

**L1 的突破 = 學會讓 AI 代勞「處理」**  
但搬運、串接、記憶都還在你身上

L1 之後，你會遇到這個問題 —

# 「這份 PDF 報告，我該用 ChatGPT 還是 NotebookLM 還 是 Claude？」

當你開始問這個問題  
你已經在往 L2 走了

Level 2

# 選工具

線 — 多個工具串成一條線

「我知道什麼時候該用什麼」

## L2 場景 — 不同素材，不同最佳工具



### PDF 論文 / 長報告

→ **NotebookLM**

深度分析、自動生成 Podcast  
摘要、跨文件比對



### 專案累積知識

→ **Claude Projects**

建知識工作空間、Custom  
Instructions、記憶累積



### 快速問答 / 搜尋

→ **Gemini**

即時回答、連網搜尋、免費  
額度最多

L2 的核心：人學會判斷。不是 AI 幫你選，而是你知道什麼時候該用什麼。

# L2 工具決策矩陣

| 素材類型   | ChatGPT | Claude       | Gemini      | NotebookLM       |
|--------|---------|--------------|-------------|------------------|
| PDF 報告 | 上傳分析    | Projects 累積  | 免費上傳        | 深度分析<br>+Podcast |
| 網頁文章   | 快速摘要    | Projects 存檔  | 連網搜尋        | 批量匯入             |
| 對話記錄   | 原生累積    | Projects 知識庫 | Gems 對話     | —                |
| 程式碼    | GPT-4o  | Artifacts 迭代 | Code Assist | —                |
| 多文件比對  | —       | Projects 多檔  | —           | 跨源頭比對            |

黃底 = 該類素材的最佳選擇。選對工具比用力寫 Prompt 更重要。



## 從點到線 — 工具串聯



### L1：點

一個工具做一件事  
每次獨立操作

### L2：線

多個工具串成流程  
人判斷接力順序

## L2 工作空間 — Claude Projects



Claude Projects = 你的「知識中繼站」。上傳文件 + 設定指令 = 有記憶的專屬 AI。

## L2 你的角色 — 指揮官



角色升級了：從搬運工 → 指揮官。你不只會用，還會**判斷**。但搬運的活，還是你在做。

## L2 能力邊界

### ✓ L2 能做

- 多工具靈活切換
- 知識工作空間（有記憶）
- 素材分流處理
- 結構化知識累積

### ✗ L2 做不到

- 工具之間自動串接
- 批量處理 100 篇文章
- 自動分類 + 自動同步
- 跨裝置即時可用

瓶頸：**工具之間不互通**。NotebookLM 的結果要手動搬到 Claude，Claude 的整理要手動存到 Notion。人還是搬運工。

## L2 突破點



**L2 → L3 的關鍵問題：**  
「我會選工具了，但能不能讓它們**自己串起來**？」

Level 3

# 創工具

## 面 — 多條線交織成自動化系統

「我造了一套自動 Pipeline !」

## L3 場景 — 我想要自動化知識管理

### ☹️ L2 的痛

每次寫筆記要手動分類  
每次要手動同步到 Notion  
換電腦就找不到  
累積 100 篇後根本不想管

### 🎯 L3 的目標

寫一篇 .md 筆記 → 存檔  
→ 自動分類、自動標籤  
→ 自動同步到 Notion  
→ 任何裝置都能查

關鍵轉變：人退出搬運迴圈。你只負責「寫」，剩下的系統自己處理。

## L3 工具 — 四層組合技

### Layer 1

#### **CLAUDE.md — AI 的工作規則**

定義專案背景、命名規範、行為約束。AI 的「員工手冊」，所有動作的基礎。

### Layer 2

#### **Skills — AI 的技能擴展**

可重複呼叫的專業能力：生成簡報、同步知識庫、品質檢查。一次定義，永久複用。

### Layer 3

#### **MCP — AI 的系統連接**

Model Context Protocol：讓 AI 串接外部服務（Notion API、GitHub、資料庫）。

### 外部

#### **GitHub Actions** + **Notion API — 自動化基礎設施**

git push 觸發 CI/CD、API 自動同步 — 這些是 MCP 連接的「對象」。

四層從內到外：規則 → 技能 → 連接 → 外部服務。Claude Code 是執行引擎，在這四層之上運作。

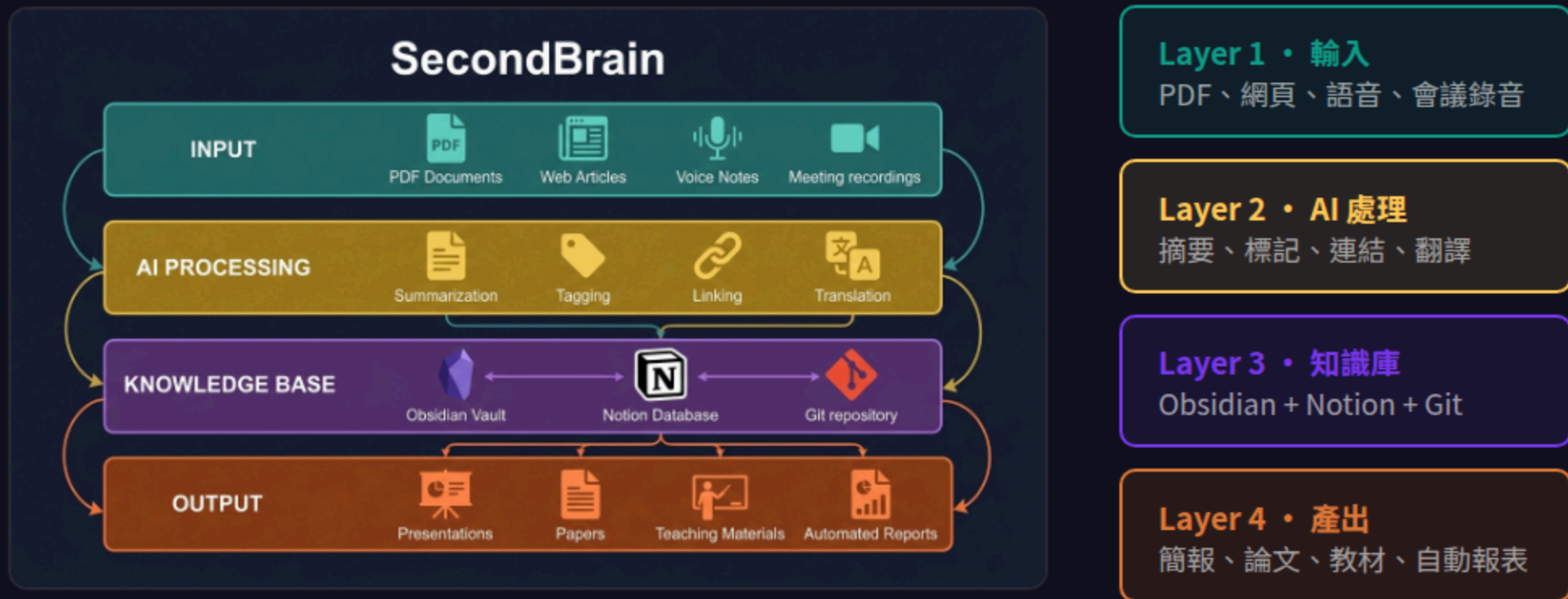


## L3 你的角色 — 建造者



角色再升級：搬運工 → 指揮官 → **建造者**。你設計一次，系統永遠替你跑。

# 實戰案例 — SecondBrain 第二大腦：架構層



380 筆知識條目，4 個 Notion 資料庫整合，7 大類自動分類 — 這是 L3 創工具的真实案例。

# 實戰案例 — SecondBrain 第二大腦：資訊流



知識不再是「存起來偶爾翻」，而是自動流動、自動產出。這就是 L3 創工具的實踐。

## SecondBrain 真實數據

380

筆知識

7

大分類

4

來源整合

~3s

增量同步時間

### 7 大知識分類

商業 / 技術 / 學習 / 投資 / 紀錄 / 研究 / 創意

### 4 個來源整合

Git 原生 (172) / 知識學習 DB (145) / 想法捕捉 (91) / Felo.ai (17)

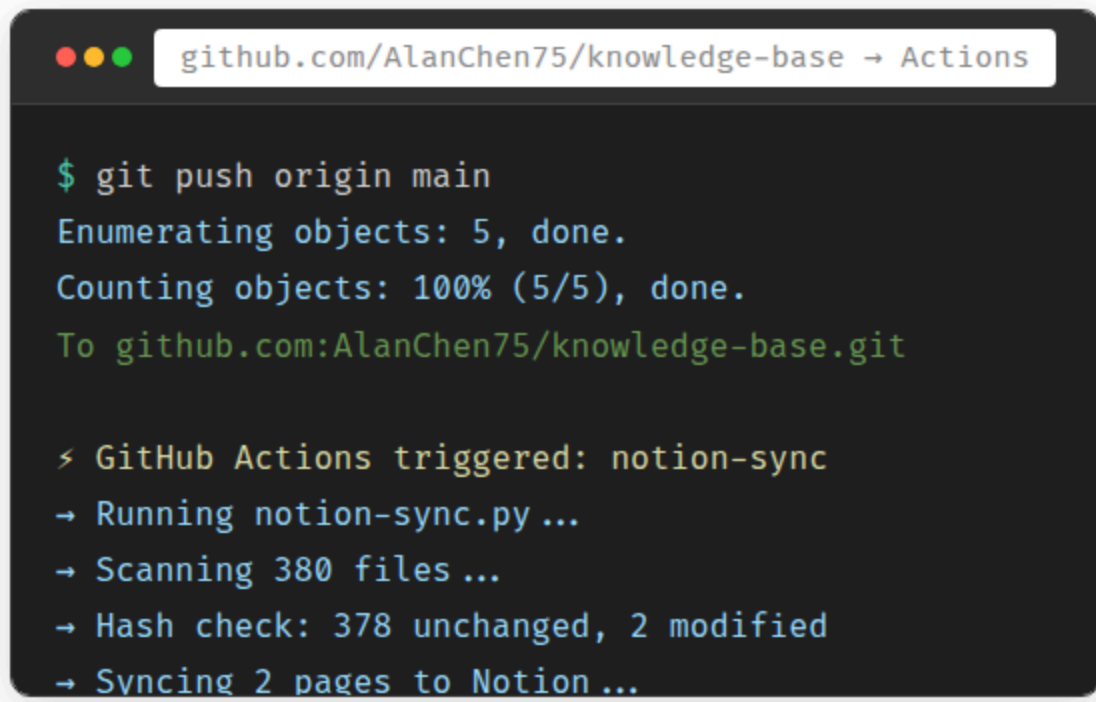
從 4 個散落的 Notion DB → 1 個統一知識庫。這就是 L3「面」的威力。

## SecondBrain — Notion 知識庫畫面



所有知識統一在一個資料庫。搜尋、篩選、排序 — 跨裝置隨時可用。

## SecondBrain — GitHub Actions 自動同步



```
github.com/AlanChen75/knowledge-base → Actions

$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
To github.com:AlanChen75/knowledge-base.git

> GitHub Actions triggered: notion-sync
→ Running notion-sync.py ...
→ Scanning 380 files ...
→ Hash check: 378 unchanged, 2 modified
→ Syncing 2 pages to Notion ...
```

你只做了 `git push`，系統自動完成：掃描 → Hash 比對 → 增量同步 → 完成。

## 從點、線到面

### L1：點

一個動作  
一個工具  
一次完成

人做所有搬運

### L2：線

多個工具  
串成流程  
人判斷接力

人還是搬運工

### L3：面

多條線交織  
工具自動串接  
系統自己跑

人退出搬運迴圈

L3 的本質：你造了一個「面」，覆蓋了從輸入到輸出的完整流程。人只負責內容創作。

## L3 能力邊界

### ✓ L3 能做

- 自動化 Pipeline
- 系統整合 (Git + API)
- 批量處理
- 跨裝置同步

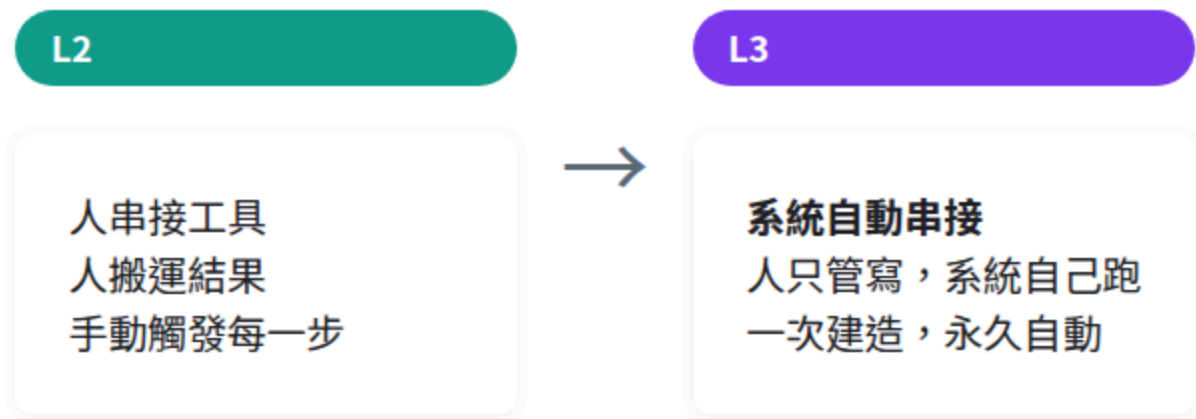
### ✗ L3 做不到

- 知識重組為新創作
- 跨情境靈活應用
- 組織級流程推廣
- 根據不同受眾調整

瓶頸：知識收了、整理了，但怎麼用？380 篇筆記，能不能變成一份新簡報、一篇新文章？



## L3 突破點



**L3 → L4 的關鍵問題：**  
「知識庫有了，能不能用它來重組創作？」

Level 4

# 流程改造

立體 — 用系統思維重新設計流程

「知識變成新的創作」

## L4 場景 — 從知識庫到新簡報

### 新任務來了

要為**不同受眾**做一份新簡報：  
不同的主題、不同的目的、不同的深度。  
你的知識庫裡有素材、有案例、有數據 —

### L3 做法 vs L4 做法

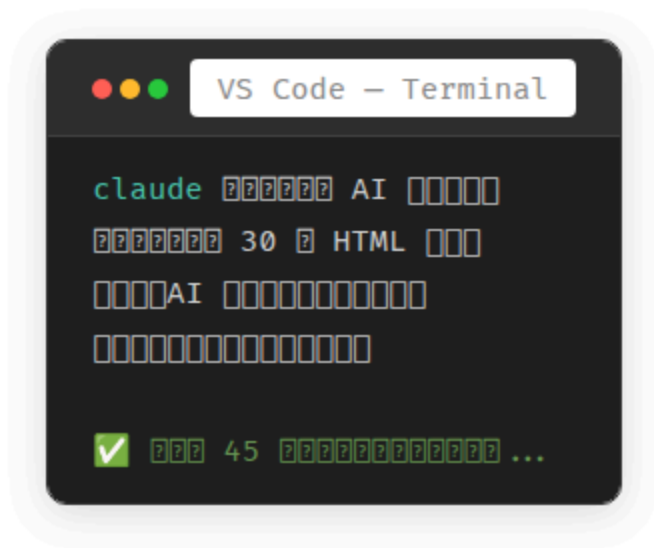
#### L3：手動翻找

打開 Notion 一篇篇找  
複製貼上到簡報  
重新排版、調整語氣

#### L4：系統化重組

用 IDE + AI 讀取知識庫  
指定受眾/目的/主題  
AI 自動重組、產出初稿

## 實戰：知識庫 × IDE × AI 重組



## 流程再設計 — AS-IS → TO-BE

### AS-IS 原本流程

1. 收到新簡報需求
2. 從零開始找資料
3. 一篇篇讀、手動摘錄
4. 排版、設計、修改
5. 重複以上（每次都重來）

🕒 每份簡報 8-16 小時



### TO-BE 改造後

1. 收到新簡報需求
2. 指定受眾 + 目的 + 主題
3. AI 從知識庫自動重組
4. 審核、微調、交付
5. 新知識回存知識庫

🕒 每份簡報 1-3 小時

知識庫不再只是「倉庫」— 它是你的「彈藥庫」。每次新任務，都是重新組裝。

## 冰山模型 — 知識管理的根因分析

### 表面：做簡報太慢

每次都要找資料、從零開始

### 結構：每次重複找資料

沒有統一的知識存取流程

### 心智模型：知識 = 用完即丟

沒有把知識當「資產」管理

### L4 改造目標

知識是可重複利用的資產 — 建立「收→存→用→回存」循環

## AI 能力矩陣 — 哪些環節最值得 AI 介入？

| 知識管理環節 | AI 介入價值 | 說明                |
|--------|---------|-------------------|
| 素材搜尋   | ● 高     | AI 從知識庫精準搜尋相關素材   |
| 內容重組   | ● 高     | 根據新受眾/目的重新架構      |
| 初稿撰寫   | ● 中高    | AI 寫初稿，人審核修正      |
| 品質審核   | ● 中     | AI 協助檢查一致性，人做最終判斷 |
| 視覺設計   | ● 中低    | 模板系統處理，創意設計仍需人    |

80/20 法則：素材搜尋 + 內容重組佔了 80% 的時間，這正是 AI 最強的地方。

## 從個人到組織 — 流程改造的擴展



同一套方法論：收 → 存 → 用 → 回存。L4 的價值是把個人的成功經驗變成可複製的流程。



## L4 能力邊界

### ✓ L4 能做

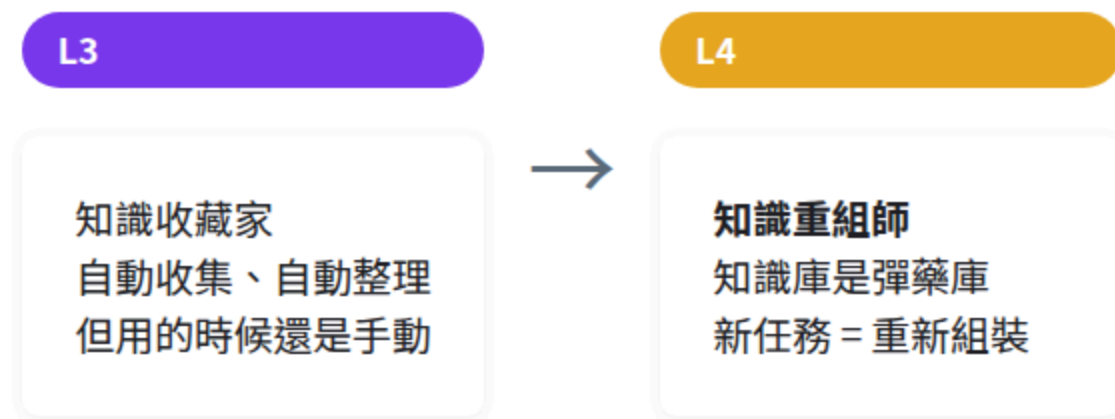
- 知識重組為新創作
- 跨情境靈活應用
- AS-IS → TO-BE 流程再造
- 個人方法擴展到組織

### ✗ L4 做不到

- 創造全新的工作方法
- AI 主動發現知識缺口
- 系統自我演化
- 定義新的遊戲規則

瓶頸：流程改善了，但**方法本身**能不能進化？能不能讓 AI 不只是工具，而是「合作夥伴」？

## L4 突破點



**L4 → L5 的關鍵問題：**  
「流程改善了，但做事的**方法本身**能不能被重新發明？」

Level 5

# 方法創新

重新定義「做事的方法」本身

「知識庫自己在長大」

## L5 不是「做得更多」 — 是「做法不同」

### L1-L4 的邏輯

用更好的工具 → 更快地做同一件事

工具升級、流程優化、效率提升  
但「做事的方法」沒變

### L5 的邏輯

重新定義「做事的方法」本身

不是更快地找知識  
而是讓知識主動找你

L4：人用 AI 管理知識 → L5：AI 主動管理知識，人做最終決策

## 傳統知識管理 vs L5 方法

| 環節 | 傳統 / L1-L4    | L5 方法創新            |
|----|---------------|--------------------|
| 收集 | 人主動找資料、存檔     | AI 自動監控、主動蒐集       |
| 分類 | 人定義分類、手動標籤    | AI 自動理解語意、動態分類     |
| 關聯 | 人記得「A 跟 B 有關」 | AI 自動發現跨領域關聯       |
| 應用 | 人搜尋、人判斷、人組裝   | AI 主動推薦「你現在可能需要這個」 |
| 演化 | 知識庫靠人維護       | 知識庫自我更新、自我優化       |

## 實例：新聞 Pipeline — AI 自主運轉



人不在迴圈內 — AI 自己蒐集、處理、發布、監控。這就是 L5 方法創新。

## 實例：論文自動收集與研究圖譜



### 傳統做法

你去找論文 → 你讀 → 你分析 → 你發現缺口  
全靠人的時間和記憶力

### L5 做法

AI 主動追蹤 → AI 分析 → AI 比對 → AI 建議  
研究者專注在最有價值的決策

## 五級全貌對比

| Level | 動作   | 工具              | 你的角色 | 知識流向     | 突破點     |
|-------|------|-----------------|------|----------|---------|
| L1    | 用工具  | ChatGPT / Gems  | 搬運工  | 人→AI→人→存 | AI 代勞處理 |
| L2    | 選工具  | 多工具+Projects    | 指揮官  | 人判斷分流    | 學會選擇判斷  |
| L3    | 創工具  | Claude Code+API | 建造者  | 系統自動串接   | 人退出搬運   |
| L4    | 流程改造 | IDE+知識庫重組       | 設計師  | 知識→新創作   | 知識成為資產  |
| L5    | 方法創新 | 自主Pipeline      | 決策者  | AI 主動推送  | 方法本身進化  |

每一級的核心不是「技術更難」，而是「**你的角色在升級**」— 搬運工→指揮官→建造者→設計師→決策者



# L5 不是更大的立體 而是改變遊戲規則

不用更努力，而是用新方法

## 方法創新的四個特徵



### ① 人退出執行迴圈

AI 自己跑完全流程  
人只在例外時介入



### ② AI 主動而非被動

不是你問 AI 才答  
而是 AI 主動告訴你需要什麼



### ③ 系統自我優化

越用越好、自我修正  
知識庫自己在成長



### ④ 方法可複製到新領域

知識管理的 L5 方法  
可以套用到任何領域的流程改造

# 你在哪一級？— 自我評估 × 學習路徑

- 1 你會用 ChatGPT / Gemini 幫你摘要、翻譯、整理嗎？  
**Yes → 你已經在 L1** 下一步：學習建立 Custom GPT / Gems 個人助手
- 2 你知道不同任務該用哪個工具嗎？（PDF → NotebookLM、專案 → Claude）  
**Yes → 你已經在 L2** 下一步：學習 Claude Projects 建知識工作空間
- 3 你有讓工具之間「自動串接」的經驗嗎？（自動化腳本、API 串接）  
**Yes → 你已經在 L3** 下一步：學習 Claude Code + CLAUDE.md 建造完整 Pipeline
- 4 你有用 AI 重新設計過工作流程嗎？（AS-IS → TO-BE）  
**Yes → 你已經在 L4** 下一步：6 週 PBL 專案實戰
- 5 你有建造過「AI 自己跑、自己優化」的系統嗎？  
**Yes → 你已經在 L5** 恭喜！你已經在創造新方法

想走完整條路？→ [aipath.cooperation.tw](https://aipath.cooperation.tw) L1 用工具（2天）→ L2 選工具（2天）→ L3 創工具（3天）  
→ L4 PBL（6週）

## Block 裁員 40% — 華爾街的狂歡

### Block 裁員 40%

將員工總數從萬餘人直接砍至 6,000 人以下  
裁員支出 4.5 至 5 億美元

核心指標

毛利潤同比增長 24% 至 28.7 億美元，超出預期  
2025 全年毛利潤達 103.6 億美元，同比增長 17%

### 股價暴漲 24%

每裁一人，市值增加 1,500 萬美元

總市值增加 330 億美金

消失的

# 4,000 人

執行層、初級分析師、冗餘的中層管理



留下的

# 6,000 人

「利用 AI 進行自動化的超級個體」

## 消失的 vs 留下的

4,000

消失的職位

### 執行層

重複性操作、資料輸入、格式處理

### 初級分析師

報表彙整、基礎數據分析、初步研究

### 冗餘的中層管理

訊息傳遞、進度追蹤、會議協調

6,000

留下的超級個體

### AI 自動化整合者

設計 AI 工作流、自動化 Pipeline

### 策略決策者

用 AI 輔助判斷，做高價值決策

### 系統架構師

設計 AI + 人的協作流程

留下的人不是「更便宜」，而是一個人 + AI = 原來十個人的產出

結束前，記住兩個詞 —

## Harness Engineering

駕馭工程

不是自己寫程式，而是駕馭 AI 來工程化。

設計規則 (CLAUDE.md)、定義技能 (Skills)、串接系統 (MCP) — 你是架構師，AI 是施工隊。

L3-L5 的核心能力。

## FDE

Forward Deployed Engineer • 前線部署工程師

不在後方寫規格，而是帶著 AI 到前線部署。

直接面對業務場景，用 AI Agent 現場解決問題、搭建系統、交付成果。

L4-L5 的實戰角色。

**Harness Engineer → FDE**：從「駕馭 AI 做事」到「帶 AI 上前線部署」— 這就是 L1 到 L5 的完整旅程。

---

# 技術不會消滅工作 但會徹底重塑 工作的性質與分佈

---

你在哪一層 Level ?  
決定你站在哪一邊。

[aipath.cooperation.tw](http://aipath.cooperation.tw)